

程序报告

学号：2312331

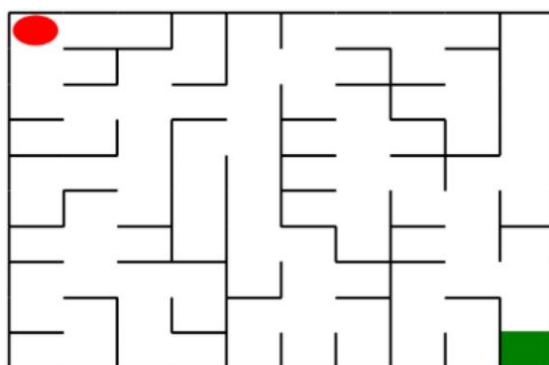
姓名：王一诺

一、问题重述

（简单描述对问题的理解，从问题中抓住主干，必填）

1.1 实验内容

在本实验中，要求分别使用基础搜索算法和 Deep QLearning 算法，完成机器人自动走迷宫。



如上图所示，左上角的红色椭圆既是起点也是机器人的初始位置，右下角的绿色方块是出口。游戏规则为：从起点开始，通过错综复杂的迷宫，到达目标点(出口)。

在任一位置可执行动作包括：向上走 'u'、向右走 'r'、向下走 'd'、向左走 'l'。

执行不同的动作后，根据不同的情况会获得不同的奖励，具体而言，有撞墙、走到出口、其余情况这三种情况。

需要您分别实现基于基础搜索算法和 Deep QLearning 算法的机器人，使机器人自动走到迷宫的出口。

1.2 实验要求

- 1 使用 Python 语言。
2. 使用基础搜索算法完成机器人走迷宫。
3. 使用 Deep QLearning 算法完成机器人走迷宫。
4. 算法部分需要自己实现，不能使用现成的包、工具或者接口。

1.3 实验环境

可以使用 Python 实现基础算法的实现，使用 Keras、PyTorch 等框架实现 Deep QLearning 算法。

1.4 注意事项

Python 与 Python Package 的使用方式，可在右侧 API 文档 中查阅。

当右上角的『Python 3』长时间指示为运行中的时候，造成代码无法执行时，可以重新启动 Kernel 解决（左上角『Kernel』-『Restart Kernel』）。

1.5 参考资料

强化学习入门 MDP: <https://zhuanlan.zhihu.com/p/25498081>

QLearning 示例: <http://mnemstudio.org/path-finding-q-learning-tutorial.htm>

QLearning 知乎解释: <https://www.zhihu.com/question/26408259>

DeepQLearning 论文: <https://files.momodel.cn/Playing Atari with Deep Reinforcement Learning.pdf>

二、设计思想

(所采用的方法, 有无对方法加以改进, 该方法有哪些优化方向(参数调整, 框架调整, 或者指出方法的局限性和常见问题), 伪代码, 理论结果验证等... **思考题, 非必填**)

1. 基础搜索算法 (DFS)

状态表示: 使用坐标 (row, col) 表示机器人在迷宫中的位置。

搜索树节点: 定义 `SearchTreeNode` 类, 包含当前节点位置 `loc`、为到达本节点所执行的动作 `to_this_action`、父节点指针 `parent`、以及子节点列表 `children`。

访问标记: 维护大小为 (height, width) 的布尔矩阵 `visited`, 记录哪些格子已被访问过, 防止重复搜索。

拓展策略: 从根节点 (起点) 开始, 将其标记为已访问。每次取当前节点 (在 DFS 中将新生成的子节点压入“栈”), 检查是否为出口, 如果是则回溯构造路径。若当前节点仍可拓展 (即为叶子且其坐标上 `visited=False`), 则调用 `expand_node()` 将所有合法 (未撞墙且未访问) 的动作对应的新节点加入并标记。若当前节点无法继续拓展, 则回溯出栈。

回溯路径: 当搜索到出口时, 从终点节点沿 `parent` 指针不断向上追溯, 收集 `to_this_action`, 逆序得到从起点到出口的动作序列。

该方法保证只要迷宫有解, DFS 能找到一条从起点到出口的路径, 时间复杂度约为 $O(N^2)$ (N 为边长), 且对内存要求较小。

2. 深度 Q-Learning 算法

状态与动作空间

状态 s : 机器人在迷宫中的坐标 (row, col)。

动作 $a \in \{u, r, d, l\}$: 分别表示向上、向右、向下、向左移动一个格子 (若撞墙则位移无效)。

Q-表结构

使用字典 `q_table`, 键为状态 s , 值为长度 4 的子字典 `{action: Q(s,a)}`。初始时, 所有 Q 值设为 0。

ϵ -贪婪策略

以概率 ϵ 随机选择一个动作 (包括会撞墙的动作), 以概率 $1-\epsilon$ 选择当前 Q 值最大的动作 (在四个方向中直接比较大小)。每次训练更新后, 将 ϵ 按比例衰减 (本实现取 $\epsilon *= 0.5$)。

奖励设计

撞墙 (`move_robot` 返回的奖励 ≤ -1): 机器人位置不变, 但获得负值惩罚; 下一状态与当前状态相同。

走到出口 (`move_robot` 返回的奖励 = `reward["destination"]`): 获得正值奖励, 标记为终止状态。

走到空白格子 (其它情况): 一般性负奖励 (在 `Maze.reward` 中预设, 通常为 -1)。

Q 值更新

每次 `train_update()`: 读取当前状态 s , 若不在 `q_table` 中则初始化 `q_table[s] = {动作: 0.0}`。依据 ϵ -贪婪策略选动作 a 。执行动作并获得即时奖励 r , 获取下一状态 $s' = \text{maze.sense_robot}()$ 。若 s' 不在 Q 表中, 初始化 `q_table[s']`。

计算更新目标: $\text{update_r} = r + \gamma * \max_{a'} Q(s', a')$ 当前 Q 值记为 $\text{current_q} = Q(s, a)$ 。

更新: $Q(s, a) = \alpha * \text{current_q} + (1 - \alpha) * (\text{update_r} - \text{current_q})$

ϵ 衰减: $\epsilon *= 0.5$ 。

测试模式 `test_update()`

仅从当前 Q 表中选出 Q 值最大的动作直接执行, 不再进行随机探索, 也不更新 Q 值, 直至走到出口或达到步数上限。

通过多轮训练, 机器人逐步学习到在各个状态下应选择的最优方向。若超参数 (α , γ , ϵ 初值) 设置合理, 且训练充分, 最终可在不同难度迷宫中走出。

三、代码内容

(能体现解题思路的主要代码, 有多个文件或模块可用多个"====="隔开, 必填)

```
=====main.py=====
# 导入相关包
import os
import random
import numpy as np
from Maze import Maze
from Runner import Runner
from QRobot import QRobot
from ReplayDataSet import ReplayDataSet
from torch_py.MinDQNRobot import MinDQNRobot as TorchRobot # PyTorch 版本
from keras_py.MinDQNRobot import MinDQNRobot as KerasRobot # Keras 版本
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim

move_map = {
    'u': (-1, 0), # up
    'r': (0, +1), # right
    'd': (+1, 0), # down
    'l': (0, -1), # left
}

def my_search(maze):
    """
    基础搜索算法示例 (深度优先搜索)
    :param maze: Maze 对象
    :return: 从起点到出口的动作序列, 例如 ['u','r','d',...]
    """
    start = maze.sense_robot()
    destination = maze.destination
    h, w, _ = maze.maze_data.shape
    visited = np.zeros((h, w), dtype=np.bool_)
```

```

def dfs(loc):
    # 如果到达终点，返回一个空的“后续动作列表”
    if loc == destination:
        return []
    visited[loc] = True
    for action in maze.can_move_actions(loc):
        dr, dc = move_map[action]
        new_loc = (loc[0] + dr, loc[1] + dc)
        if not visited[new_loc]:
            subpath = dfs(new_loc)
            if subpath is not None:
                return [action] + subpath
    return None
path = dfs(start)
return path if path is not None else []

class Robot(QRobot):
    valid_action = ['u', 'r', 'd', 'l']
    def __init__(self, maze, alpha=0.5, gamma=0.9, epsilon=0.5):
        self.maze = maze
        self.state = None
        self.action = None
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon # 动作随机选择概率
        self.q_table = {}
        self.maze.reset_robot() # 重置机器人状态
        self.state = self.maze.sense_robot() # state 为机器人当前状态

        if self.state not in self.q_table: # 如果当前状态不存在，则为 Q 表添加新列
            self.q_table[self.state] = {a: 0.0 for a in self.valid_action}

    def train_update(self):
        self.state = self.maze.sense_robot() # 获取机器人当初所处迷宫位置
        # 如果当前状态不在 Q 表，则添加
        if self.state not in self.q_table:
            self.q_table[self.state] = {a: 0.0 for a in self.valid_action}
        #  $\epsilon$ -贪婪：以  $\epsilon$  概率随机选动作，否则选 Q 值最大的动作（四个方向均可选，
        # 即使会撞墙）
        if random.random() < self.epsilon:
            action = random.choice(self.valid_action)
        else:
            action = max(self.q_table[self.state], key=self.q_table[self.state].get)
        # 执行动作，返回奖励（撞墙会返回相应惩罚并让机器人留在原地）

```

```

reward = self.maze.move_robot(action)
next_state = self.maze.sense_robot() # 执行动作后的位置
# 如果 next_state 不在 Q 表, 则添加
if next_state not in self.q_table:
    self.q_table[next_state] = {a: 0.0 for a in self.valid_action}
# 计算更新值: reward +  $\gamma$  * max_a' Q(next_state, a')
update_r = reward + self.gamma * float(max(self.q_table[next_state].values()))
current_q = self.q_table[self.state][action]
# 更新 Q 值
self.q_table[self.state][action] = (
    self.alpha * current_q
    + (1 - self.alpha) * (update_r - current_q)
)
# 衰减  $\epsilon$ 
self.epsilon *= 0.5
return action, reward

def test_update(self):
    self.state = self.maze.sense_robot() # 获取机器人现在所处迷宫位置
    # 如果当前状态不在 Q 表, 则添加
    if self.state not in self.q_table:
        self.q_table[self.state] = {a: 0.0 for a in self.valid_action}
    # 测试阶段直接选 Q 值最大的动作 (四个方向均可选)
    action = max(self.q_table[self.state], key=self.q_table[self.state].get)
    reward = self.maze.move_robot(action) # 执行动作并返回奖励
    return action, reward

```

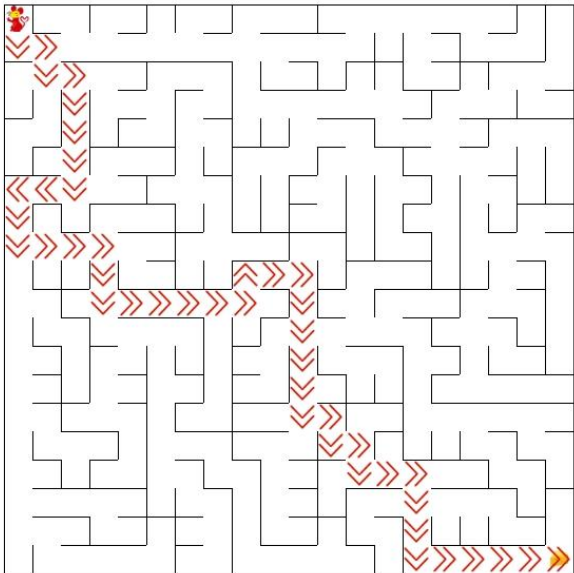
四、实验结果

(实验结果, 必填)

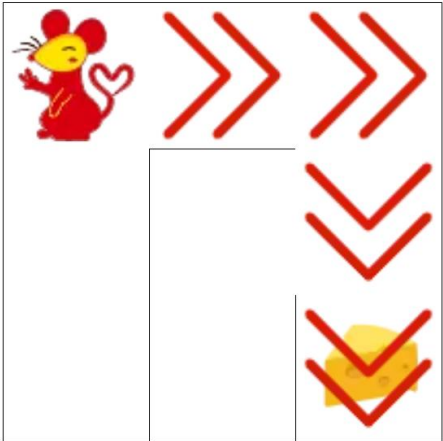
=====

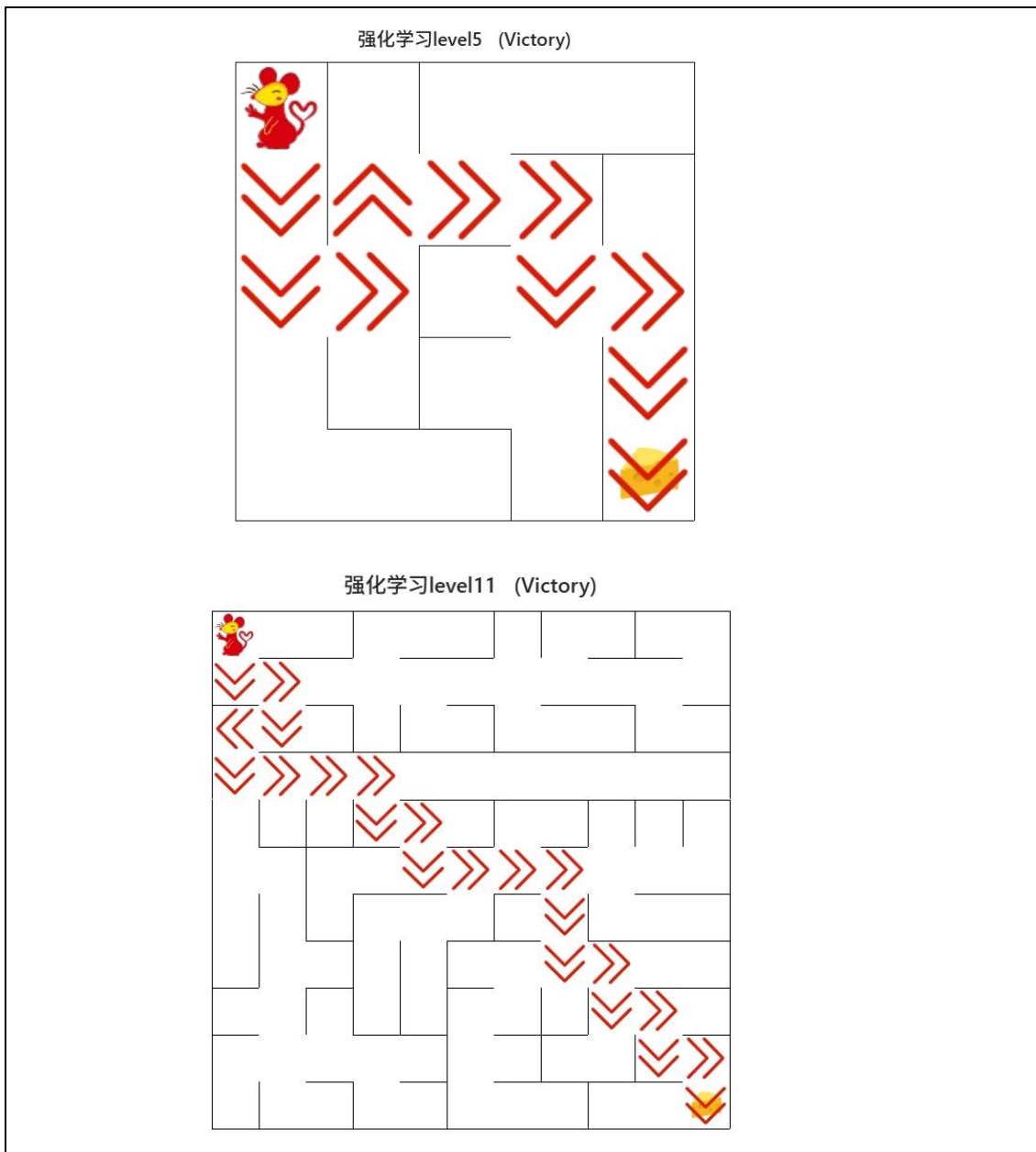
测试点	状态	时长	结果
测试基础搜索算法	✓	1s	恭喜, 完成了迷宫
测试强化学习算法(中级)	✓	1s	恭喜, 完成了迷宫
测试强化学习算法(初级)	✓	2s	恭喜, 完成了迷宫
测试强化学习算法(高级)	✓	2s	恭喜, 完成了迷宫

基础搜索算法 (Victory)



强化学习level3 (Victory)





五、总结

（自评分析（是否达到目标预期，可能改进的方向，实现过程中遇到的困难，从哪些方面可以提升性能，模型的超参数和框架搜索是否合理等），**思考题，非必填**）

通过本次实验，对基础搜索算法和 QLearning 算法有了一定的认知和了解。初期 Q 表更新公式写法不符合题目要求，导致训练完毕后还会不断掉头原路返回，修正更新公式就可以了。

两种算法各有优劣：基础搜索（DFS）实现简单、速度快，能够保证找到一条可行路径。但无法对路径长度做最优保证，也不适合动态环境；深度 Q-Learning 通过与环境的反复交互，机器人能够学会一套近似最优的策略，应对多种迷宫格局。Q-Learning 方法对于中小规模迷宫能在有限训练后收敛，但当迷宫规模扩展至较大时，状态空间过大，Q 表可能爆炸，收敛缓慢。